Fullstack Web Development

Session 4 | Week 4/Day 2 | 120 min

Javascript - Part 03 - The End

Instructor: Ms. Liev Nova

090667393

Web APIs

What is a Web API?

A **Web API (Application Programming Interface)** allows JavaScript to **interact with the browser or external services**

Web API = tools provided by browser

Types of Web APIs

- **Browser APIs** → work inside browser
- **Server APIs** → communicate with server

Browser APIs

All browsers provide built-in APIs to help developers perform complex tasks.

Example: Geolocation API — Get user's location

```
navigator.geolocation.getCurrentPosition(position => {
    console.log(position.coords.latitude);
    console.log(position.coords.longitude);
});
```

Browser APIs

Web Form API (Validation)

Used to validate user input in forms

Constraint Validation Methods

Method	Description
checkValidity()	Returns true if input is valid
setCustomValidity()	Set custom error message

Constraint Validation Properties

Property	Description
validity	Contains validation status
validationMessage	Error message
willValidate	Whether input will be validated

Browser APIs

Web Form API (Validation)

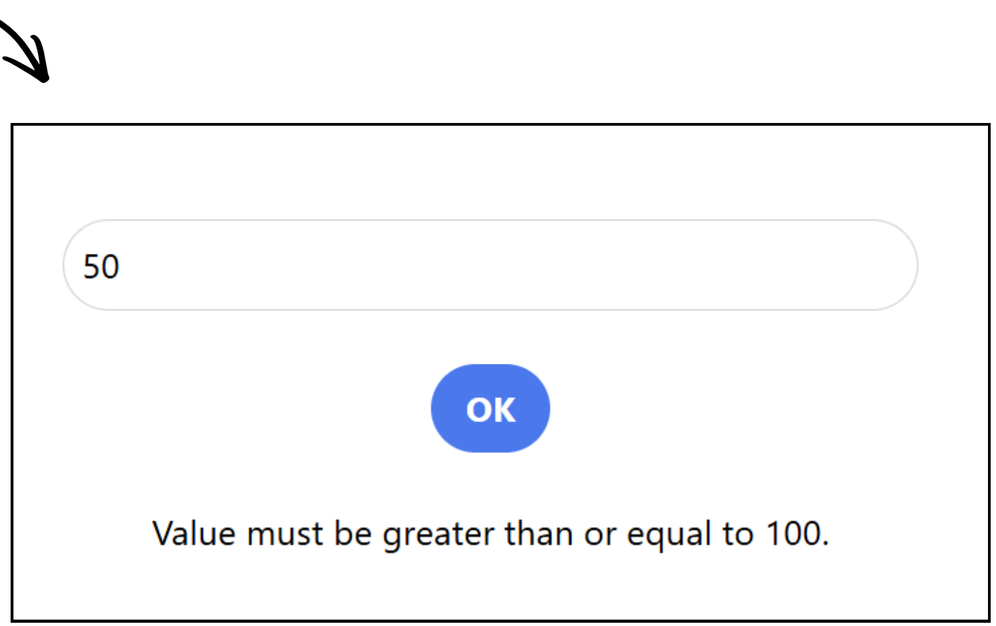
Example 1 (Form Validation)

```
<input id="id1" type="number" min="100" max="300" required>
<button onclick="myFunction()">OK</button>

<p id="demo"></p>

<script>
    function myFunction() {
        const inpObj = document.getElementById("id1");

        if (!inpObj.checkValidity()) {
            document.getElementById("demo").textContent = inpObj.validationMessage;
        }
    }
</script>
```



Browser APIs

Web Form API (Validation)

Example 2 (Range Validation)

```
<input id="id1" type="number" max="100">
<button onclick="myFunction()">OK</button>

<p id="demo"></p>

<script>
    function myFunction() {
        let text = "Value OK";

        let input = document.getElementById("id1");

        if (input.validity.rangeOverflow) {
            text = "Value too large";
        }

        document.getElementById("demo").textContent = text;
    }
</script>
```

Browser APIs

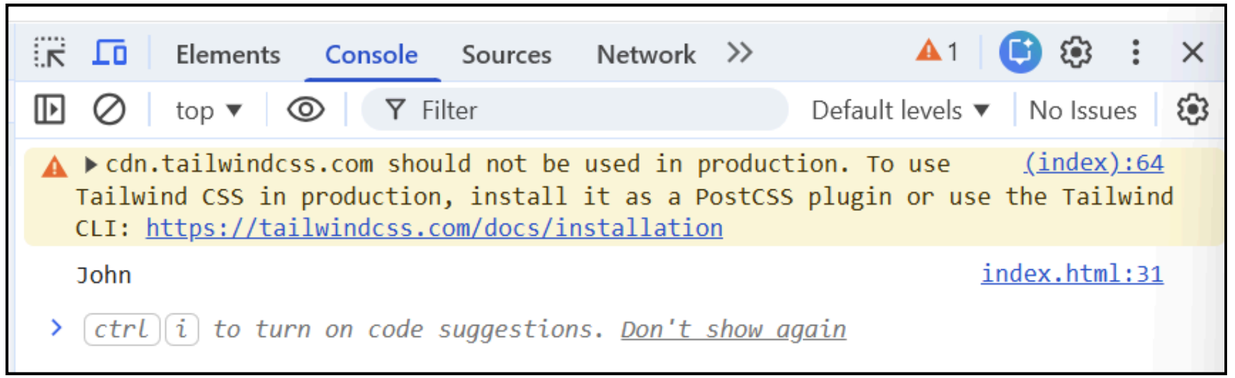
Web Storage API

Used to store data in the browser

localStorage

- Stores data permanently
- No expiration

```
localStorage.setItem("name", "John");
console.log(localStorage.getItem("name"));
```



Browser APIs

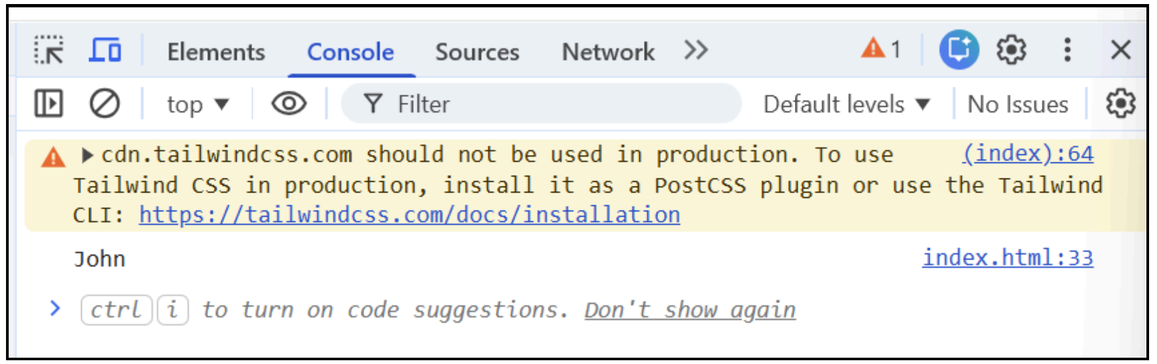
Web Storage API

Used to store data in the browser

sessionStorage

- Stores data **temporarily**
- Deleted when browser closes

```
sessionStorage.setItem("name", "John");
console.log(sessionStorage.getItem("name"));
```



Browser APIs

Geolocation API — Used to get user's location

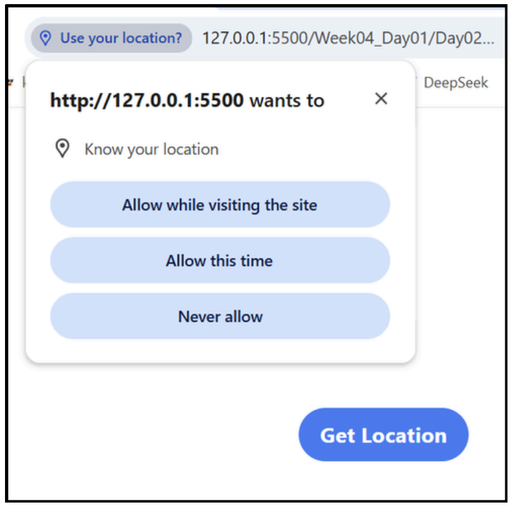
```
<button onclick="getLocation()">Get Location</button>
    <p id="demo"></p>

    <script>

        const x = document.getElementById("demo");

        function getLocation() {
            if (navigator.geolocation) {
                navigator.geolocation.getCurrentPosition(showPosition);
            } else {
                x.textContent = "Geolocation is not supported.";
            }
        }

        function showPosition(position) {
            x.innerHTML =
                "Latitude: " + position.coords.latitude +
                "<br>Longitude: " + position.coords.longitude;
        }
    </script>
```



Important:

- User must **allow permission**
- May not work in HTTP (needs HTTPS)

Browser APIs

Fetch API — Used to request data from a server

Example:

```
fetch("data.txt")
.then(response => response.text())
.then(data => console.log(data));
```

Example (JSON)

```
fetch("https://jsonplaceholder.typicode.com/users")
.then(res => res.json())
.then(data => console.log(data));
```

Request APIs

Fetch API vs Axios

When a client application wants data from a server, it sends an **HTTP request.**

The server processes the request and sends back an **HTTP response.**

Basic Flow:

Request APIs

Fetch API vs Axios

The response usually contains:

- resource data
- status code
- headers
- message

Example JSON response:

```
{
  "id": 1,
  "name": "John",
  "email": "john@example.com"
}
```

Request APIs

What is Fetch API?

Fetch API is a built-in browser API used to make HTTP requests.

It can:

- request data from a server
- send data to a server
- work with JSON, text, and other formats

Example:

```
fetch("https://jsonplaceholder.typicode.com/users")
    .then(res => res.json())
    .then(data => console.log(data));
```

Request APIs

What is Axios?

Axios is a popular external JavaScript library used to make HTTP requests.

It provides:

- simpler syntax
- automatic JSON parsing
- easier error handling
- more advanced features

Example:

```
axios.get("https://jsonplaceholder.typicode.com/users/1")
        .then(response => console.log(response.data))
        .catch(error => console.log("Error:", error));
```

Request APIs

Fetch API vs Axios

Feature	Fetch API	Axios
Built-in	Yes	No, external library
JSON Parsing	Manual with response.json()	Automatic with response.data
HTTP Error Handling	Manual check with response.ok	Automatic for bad status
Code Length	Longer	Shorter
Extra Features	Limited	Many features
Interceptors	No	Yes
Request Cancellation	Limited / more manual	Easier
Browser Support	Native in modern browsers	Requires library import

Request APIs

Key Differences

1. Built-in Support

Fetch API — Already available in modern browsers.

```
fetch("https://api.example.com/data")
```

Axios — Must be installed or included first.

```
<script src="https://cdn.jsdelivr.net/npm/axios/dist/axios.min.js"></script>
```

Request APIs

Key Differences

2. JSON Parsing

Fetch API — You must manually convert the response.

```
fetch("https://api.example.com/data")
    .then(response => response.json())
    .then(data => console.log(data));
```

Axios — Response data is already parsed.

```
axios.get("https://api.example.com/data")
        .then(response => console.log(response.data));
```

Request APIs

Key Differences

3. Error Handling

Fetch API — Fetch does not throw an error for HTTP status like 404 or 500 automatically. You need to check response.ok.

```
fetch("https://api.example.com/data")
    .then(response => {
        if (!response.ok) {
            throw new Error("HTTP error");
        }
        return response.json();
    })
    .then(data => console.log(data))
    .catch(error => console.log("Error:", error));
```

Request APIs

Key Differences

3. Error Handling

Axios — Axios automatically throws an error for bad HTTP status.

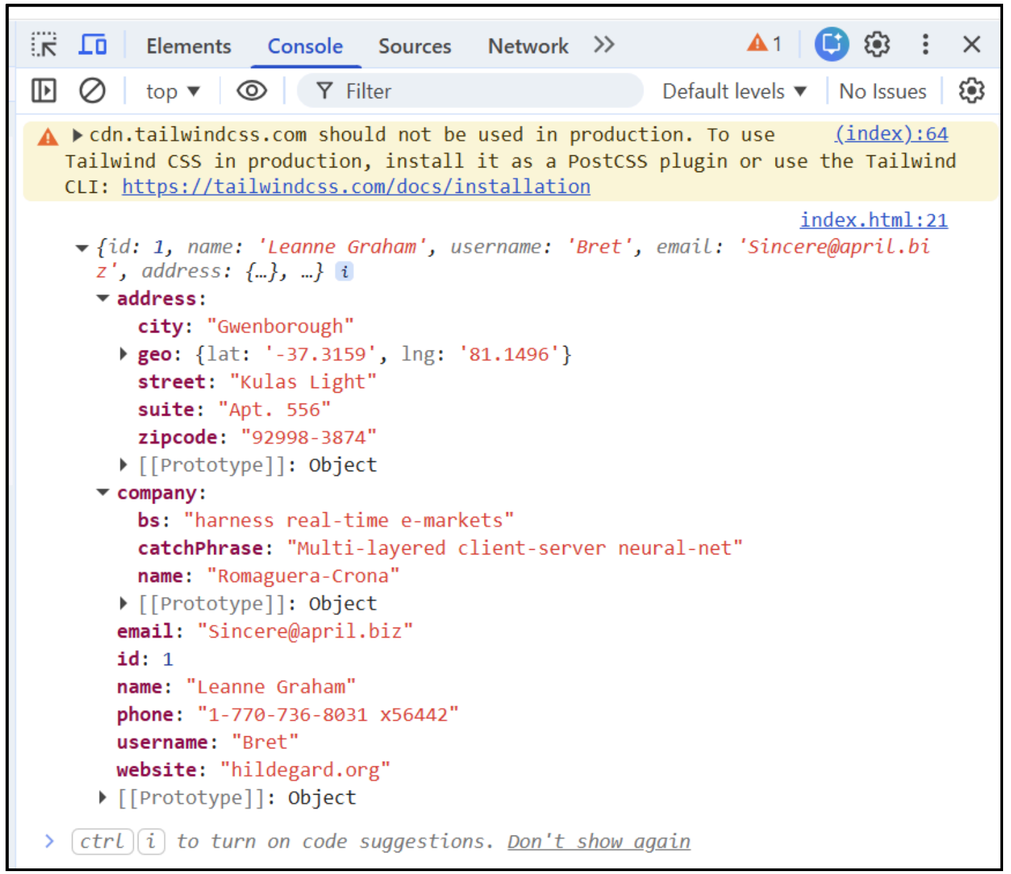
```
axios.get("https://api.example.com/data")
        .then(response => console.log(response.data))
        .catch(error => console.log("Error:", error));
```

Request APIs

Output Example

Fetch API Output

```
fetch("https://jsonplaceholder.typicode.com/users/1")
    .then(response => response.json())
    .then(data => console.log(data));
```

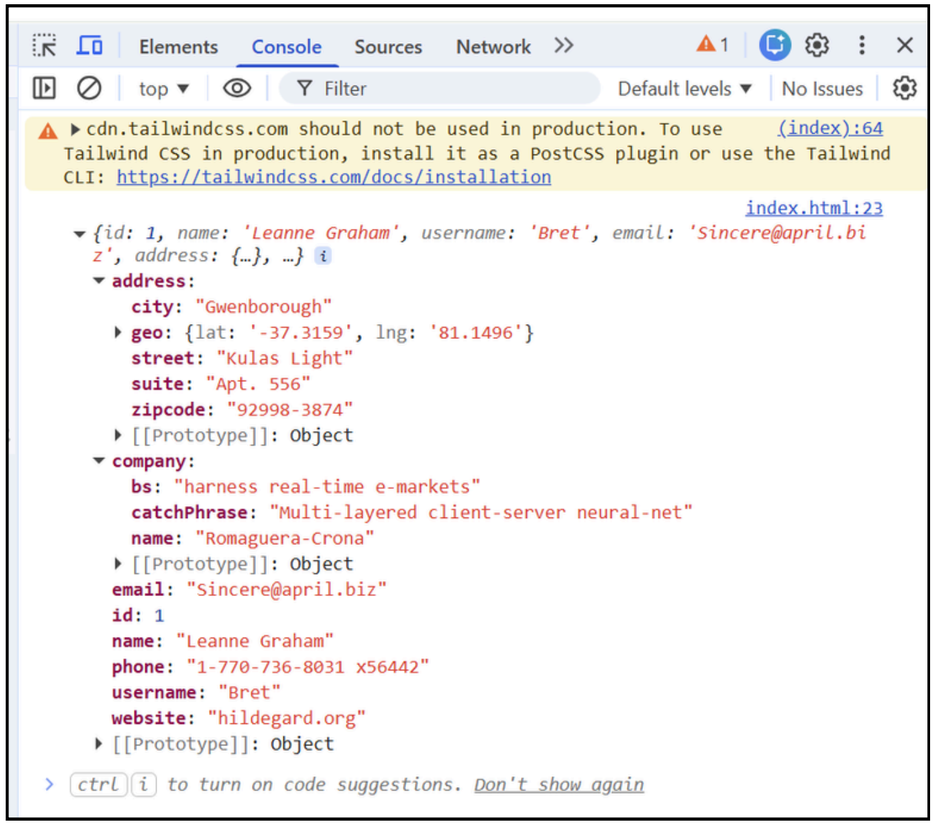


Request APIs

Output Example

Axios Output

```
axios.get("https://jsonplaceholder.typicode.com/users/1")
        .then(response => console.log(response.data));
```



Request APIs

Advantages of Fetch API

- built into browser
- no installation needed
- modern standard API
- good for simple projects

Advantages of Axios

- easier to write
- automatic JSON parsing
- easier error handling
- supports interceptors
- supports request/response customization
- useful in large projects

JSON

What is JSON?

JSON stands for **JavaScript Object Notation**

It is a **lightweight format** used to:

- store data
- send data between client and server

JSON = text format for data

Why use JSON?

- easy to read
- easy to write
- fast to parse
- widely used in APIs

JSON

JSON Example — Employees Data

```
{
  "employees": [
    { "firstName": "John", "lastName": "Doe" },
    { "firstName": "Anna", "lastName": "Smith" },
    { "firstName": "Peter", "lastName": "Jones" }
  ]
}
```

- {} → object
- [] → array
- "key": value → property

JSON

JSON Syntax Rules

- Data is in **name/value pairs**
- Data is separated by **commas**
- Objects use {}
- Arrays use []

Important Rule — Keys must be in **double quotes**

```
{ name: "John" }
```

Wrong

```
{ "name": "John" }
```

Correct

JSON

Valid Data Types

JSON supports:

- string
- number
- object
- array
- boolean
- null

Example:

```
{
  "name": "John",
  "age": 25,
  "isStudent": false,
  "skills": ["JS", "HTML"],
  "address": { "city": "Phnom Penh" },
  "phone": null
}
```

JSON

JSON vs JavaScript Object

- JSON (String format)

```
let json = '{"name":"John","age":25}';
```

- JavaScript Object

```
let obj = { name: "John", age: 25 };
```

Difference:

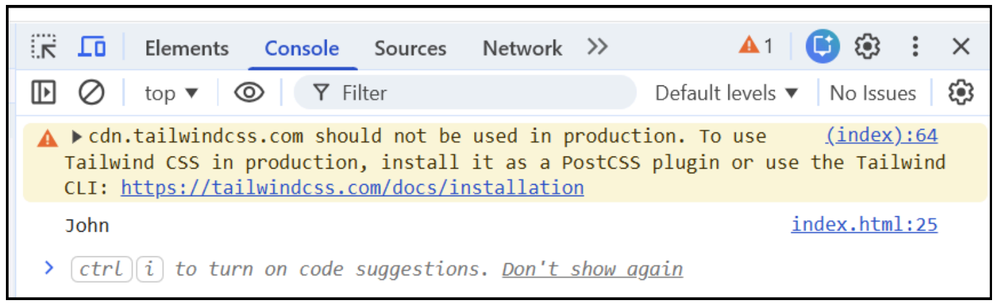
- JSON = text
- Object = real JavaScript data

JSON

JSON.parse()

Convert JSON → JavaScript Object

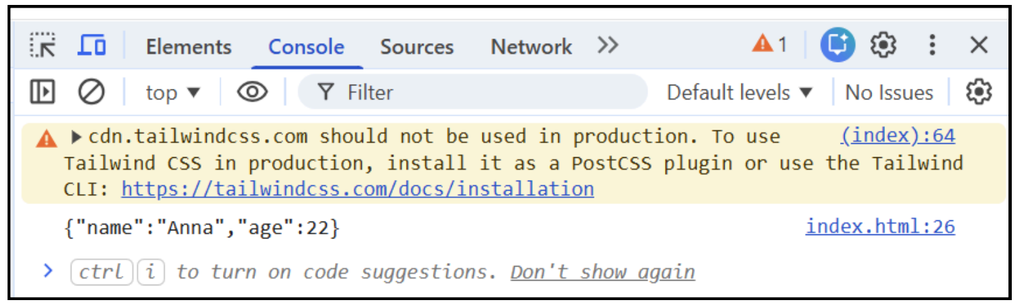
```
let json = '{"name":"John","age":25}';
let obj = JSON.parse(json);
console.log(obj.name);
```



JSON.stringify()

Convert Object → JSON

```
let obj = { name: "Anna", age: 22 };
let json = JSON.stringify(obj);
console.log(json);
```



JSON

Why JSON is Better than XML?

- JSON = modern
- XML = old and complex

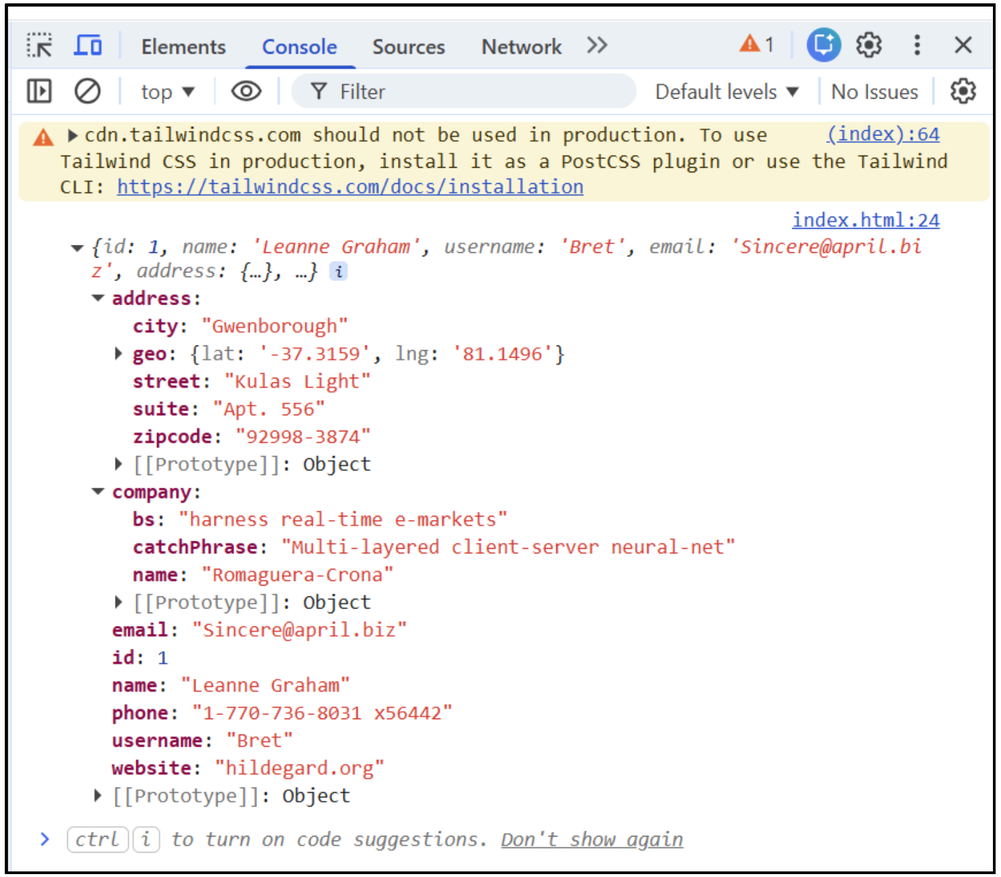
JSON	XML
Easy to read	Hard to read
Less code	More code
Faster	Slower
Directly usable in JS	Needs parsing

JSON

Real Use Case (API)

```
fetch("https://jsonplaceholder.typicode.com/users/1")
    .then(res => res.json())
    .then(data => console.log(data));
```

- JSON = data format
- parse() → JSON → Object
- stringify() → Object → JSON



JSON

Common Mistakes

- Missing quotes on keys
- Using single quotes
- Trailing comma

Example:

```
{
  "name": "John" ,
}
```

Wrong

```
{
  "name": "John"
}
```

Correct

THANK YOU

For your attention !